Building and publishing a package

uv supports building Python packages into source and binary distributions via `uv build` and uploading them to a registry with `uv publish`.

Preparing your project

Before attempting to publish your project, you'll want to make sure it's ready to be packaged for distribution.

If your project does not include a `[build-system]` definition in the `pyproject.toml`, uv will not build it during `uv sync` operations in the project, but will fall back to the legacy setuptools build system during `uv build`.

!!! note

```
Projects created with `uv init` include a `[build-system]` definition by default.
```

We strongly recommend configuring a build system. Read more about build systems in the project configuration documentation.

Building your package

Build your package with `uv build`:

```
$ uv build
```

By default, `uv build` will build the project in the current directory, and place the built artifacts in a `dist/` subdirectory.

Alternatively, `uv build <SRC>` will build the package in the specified directory, while `uv build --package <PACKAGE>` will build the specified package within the current workspace.

!!! info

```
By default, `uv build` respects `tool.uv.sources` when resolving build dependencies
from the
`build-system.requires` section of the `pyproject.toml`. When publishing a package,
we recommend
running `uv build --no-sources` to ensure that the package builds correctly when
`tool.uv.sources`
is disabled, as is the case when using other build tools, like [`pypa/build`]
(https://github.com/pypa/build).
```

Updating your version

The `uv version` command provides conveniences for updating the version of your package before you publish it. See the project docs for reading your package's version.

To update to an exact version, provide it as a positional argument:

```
$ uv version 1.0.0
hello-world 0.7.0 => 1.0.0
```

To preview the change without updating the `pyproject.toml`, use the `--dry-run` flag:

```
$ uv version 2.0.0 --dry-run
hello-world 1.0.0 => 2.0.0
$ uv version
hello-world 1.0.0
```

To increase the version of your package semantics, use the `--bump` option:

```
$ uv version --bump minor
hello-world 1.2.3 => 1.3.0
```

The `--bump` option supports the following common version components: `major`, `minor`, `patch`, `stable`, `alpha`, `beta`, `rc`, `post`, and `dev`. When provided more than once, the components will be applied in order, from largest (`major`) to smallest (`dev`).

You can optionally provide a numeric value with `--bump <component>=<value>` to set the resulting component explicitly:

```
$ uv version --bump patch --bump dev=66463664
hello-world 0.0.1 => 0.0.2.dev66463664
```

To move from a stable to pre-release version, bump one of the major, minor, or patch components in addition to the pre-release component:

```
$ uv version --bump patch --bump beta
hello-world 1.3.0 => 1.3.1b1
$ uv version --bump major --bump alpha
hello-world 1.3.0 => 2.0.0a1
```

When moving from a pre-release to a new pre-release version, just bump the relevant pre-release component:

```
$ uv version --bump beta
hello-world 1.3.0b1 => 1.3.0b2
```

When moving from a pre-release to a stable version, the `stable` option can be used to clear the pre-release component:

```
$ uv version --bump stable
hello-world 1.3.1b2 => 1.3.1
```

!!! info

```
By default, when `uv version` modifies the project it will perform a lock and sync.
To
prevent locking and syncing, use `--frozen`, or,  to just prevent syncing, use `--no-
sync`.
```

Publishing your package

!!! note

```
A complete guide to publishing from GitHub Actions to PyPI can be found in the
[GitHub Guide](integration/github.md#publishing-to-pypi)
```

Publish your package with `uv publish`:

```
$ uv publish
```

Set a PyPI token with `--token` or `UV_PUBLISH_TOKEN`, or set a username with `--username` or `UV_PUBLISH_USERNAME` and password with `--password` or `UV_PUBLISH_PASSWORD`. For publishing to PyPI from GitHub Actions or another Trusted Publisher, you don't need to set any credentials. Instead, add a trusted publisher to the PyPI project.

!!! note

```
PyPI does not support publishing with username and password anymore, instead you need
to
generate a token. Using a token is equivalent to setting `--username __token__` and
```

```
using the
token as password.
```

If you're using a custom index through `[[tool.uv.index]]`, add `publish-url` and use `uv publish --index <name>`. For example:

```toml
[[tool.uv.index]]
name = "testpypi"
url = "https://test.pypi.org/simple/"
publish-url = "https://test.pypi.org/legacy/"
explicit = true
```

!!! note

```
When using `uv publish --index <name>`, the `pyproject.toml` must be present, i.e.,
you need to
have a checkout step in a publish CI job.
```

Even though `uv publish` retries failed uploads, it can happen that publishing fails in the middle, with some files uploaded and some files still missing. With PyPI, you can retry the exact same command, existing identical files will be ignored. With other registries, use `--check-url <index url>` with the index URL (not the publishing URL) the packages belong to. When using `--index`, the index URL is used as check URL. uv will skip uploading files that are identical to files in the registry, and it will also handle raced parallel uploads. Note that existing files need to match exactly with those previously uploaded to the registry, this avoids accidentally publishing source distribution and wheels with different contents for the same version.

Uploading attestations with your package

!!! note

```
Some third-party package indexes may not support attestations, and may
reject uploads that include them (rather than silently ignoring them).
If you encounter issues when uploading, you can use `--no-attestations` or
`UV_PUBLISH_NO_ATTESTATIONS` to disable uv's default behavior.
```

!!! tip

```
`uv publish` does not currently generate attestations; attestations must
be created separately before publishing.
```

`uv publish` supports uploading attestations to registries that support them, like PyPI.

uv will automatically discover and match attestations. For example, given the following `dist/` directory, `uv publish` will upload the attestations along with their corresponding distributions:

```
$ ls dist/
hello_world-1.0.0-py3-none-any.whl
hello_world-1.0.0-py3-none-any.whl.publish.attestation
hello_world-1.0.0.tar.gz
hello_world-1.0.0.tar.gz.publish.attestation
```

Installing your package

Test that the package can be installed and imported with `uv run`:

```
$ uv run --with <PACKAGE> --no-project -- python -c "import <PACKAGE>"
```

The `--no-project` flag is used to avoid installing the package from your local project directory.

!!! tip

```
If you have recently installed the package, you may need to include the
`--refresh-package <PACKAGE>` option to avoid using a cached version of the package.
```

Next steps

To learn more about publishing packages, check out the PyPA guides on building and publishing.

Or, read on for guides on integrating uv with other software.